

Inheritance in Python

A Comprehensive Guide

Slide 1: Introduction

What is Inheritance?

- Inheritance allows a class (child/derived) to acquire properties and behaviors from another class (parent/base)
- Promotes **code reuse** — write once, use many times
- Establishes an **"is-a" relationship**: a Dog is an Animal
- Python supports single, multiple, and multilevel inheritance

```
class Animal:          # Base class
    pass

class Dog(Animal):     # Derived class – Dog inherits from Animal
    pass
```

Slide 2: Why Use Inheritance?

Benefits of Inheritance:

- **Code Reusability** — Common logic in one place
- **Extensibility** — Add features without modifying existing code
- **Logical Hierarchy** — Models real-world relationships
- **Polymorphism** — Treat derived objects as base type



Slide 3: Basic Inheritance Syntax

Simple inheritance:

```
class Parent:
    def __init__(self, name):
```

```
        self.name = name

    def greet(self):
        return f"Hello, I'm {self.name}"

class Child(Parent):    # Child inherits from Parent
    pass                # No new code – all inherited

c = Child("Alice")
print(c.greet())        # Hello, I'm Alice
print(c.name)           # Alice
```

The child class has access to all methods and attributes of the parent.

Slide 4: The **object** Base Class

Every class in Python inherits from **object:**

```
class A:                # Implicitly: class A(object):
    pass

class B(object):        # Explicit – same thing
    pass

print(A.__bases__)      # (<class 'object'>,)
```

- **object** provides default implementations of `__str__`, `__repr__`, `__eq__`, etc.
 - Even if you don't specify a parent, inheritance is always happening
-

Slide 5: Inheriting Attributes

Constructors and attributes propagate down:

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

class Car(Vehicle):
    def __init__(self, brand, model):
        Vehicle.__init__(self, brand)    # Explicit parent call
        self.model = model

c = Car("Toyota", "Camry")
print(c.brand)    # Toyota – from parent
print(c.model)    # Camry – own attribute
```

Without calling the parent's `__init__`, `brand` would never be set.

Slide 6: The `super()` Function

Better way to call parent methods:

```
class Car(Vehicle):
    def __init__(self, brand, model):
        super().__init__(brand)    # No need to name Vehicle
        self.model = model
```

- `super()` returns a proxy object that delegates to the parent class
 - No need to hardcode the parent class name
 - Works correctly with multiple inheritance (MRO)
 - Preferred pattern in modern Python
-

Slide 7: Method Overriding

Redefining a parent method in the child:

```
class Animal:
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):          # Override parent's speak()
        return "Woof!"

class Cat(Animal):
    def speak(self):          # Override parent's speak()
        return "Meow!"

d = Dog()
c = Cat()
print(d.speak())    # Woof!
print(c.speak())    # Meow!
```

Same method name, different behavior per class — **polymorphism**.

Slide 8: Calling Parent's Overridden Method

Using `super()` in overridden methods:

```
class Employee:
    def __init__(self, emp_id, name):
        self.emp_id = emp_id
        self.name = name

    def info(self):
        return f"ID: {self.emp_id}, Name: {self.name}"

class Manager(Employee):
    def __init__(self, emp_id, name, team_size):
        super().__init__(emp_id, name)  # Parent constructor
        self.team_size = team_size

    def info(self):  # Override
        return super().info() + f", Team: {self.team_size} people"
```

Slide 9: Overriding vs Overloading

Important distinction:

Concept	Python	C++/Java
Overriding	Child redefines parent method	Same signature, different body
Overloading	Not directly supported	Multiple methods with same name, different params

```
# Python: No true overloading – last definition wins
class Example:
    def show(self, a):
        print(a)
    def show(self, a, b):  # Overwrites previous
        print(a, b)
```

Use default parameters or `*args` as workarounds.

Slide 10: Extending vs Overriding

Two ways to modify parent behavior:

```
class Base:
    def process(self):
        print("Base processing")

class Extend(Base):
    def process(self):
        super().process()  # Call parent FIRST
```

```
print("Extra processing") # Then add more
# Order: Base → Extra

class Override(Base):
    def process(self):
        print("Complete replacement") # Parent not called
```

- **Extending:** call `super()` then add logic
- **Overriding:** replace completely (no `super()` call)

Slide 11: Constructor Call Order — Single Inheritance

When a derived object is created:

```
class A:
    def __init__(self):
        print("A.__init__ called")

class B(A):
    def __init__(self):
        print("B.__init__ called")
        super().__init__()

b = B()
# Output:
# B.__init__ called
# A.__init__ called
```

Order: **Child** → **Parent** (if `super()` is called in child) Without `super()` in child: only Child's constructor runs.

Slide 12: Constructor Call Order — Multilevel

```
class Grandparent:
    def __init__(self):
        print("Grandparent.__init__")

class Parent(Grandparent):
    def __init__(self):
        print("Parent.__init__")
        super().__init__()

class Child(Parent):
    def __init__(self):
        print("Child.__init__")
        super().__init__()

c = Child()
```

```
# Output:  
# Child.__init__  
# Parent.__init__  
# Grandparent.__init__
```

Each level calls `super().__init__()`, creating a chain: **Child** → **Parent** → **Grandparent**.

Slide 13: Constructor Call Order — Multiple Inheritance

```
class A:  
    def __init__(self):  
        print("A.__init__")  
        super().__init__()  
  
class B:  
    def __init__(self):  
        print("B.__init__")  
        super().__init__()  
  
class C(A, B):  
    def __init__(self):  
        print("C.__init__")  
        super().__init__()  
  
c = C()  
# Output:  
# C.__init__  
# A.__init__  
# B.__init__
```

Why does `A.__init__` call `B.__init__`? Because of MRO.

Slide 14: MRO — Method Resolution Order

The C3 Linearization Algorithm:

```
class C(A, B): pass  
print(C.__mro__)  
# (<class 'C'>, <class 'A'>, <class 'B'>, <class 'object'>)
```

MRO determines:

1. Which method runs when called on an object
2. The order `super()` traverses through

Rules:

- A child comes before its parent
- Parents are visited in declaration order
- `object` is always last

Slide 15: MRO — Diamond Problem

Python solves the diamond problem via MRO:

```
class A:
    def method(self):
        print("A.method")

class B(A):
    def method(self):
        print("B.method")
        super().method()

class C(A):
    def method(self):
        print("C.method")
        super().method()

class D(B, C):
    def method(self):
        print("D.method")
        super().method()

d = D()
d.method()
# Output:
# D.method
# B.method
# C.method
# A.method
```



MRO for D: $D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$

Slide 16: Viewing MRO

How to inspect MRO:

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

# Three ways to see MRO:
print(D.__mro__)
print(D.mro())
import inspect
print(inspect.getmro(D))

# Note: D.__mro__ and inspect.getmro(D) return a tuple
#       D.mro() returns a list – content matches, type differs
```

Useful for debugging complex inheritance hierarchies.

Slide 17: `super()` Without Arguments (Python 3)

Simplified syntax:

```
class A:
    def __init__(self):
        super().__init__()    # No arguments needed!
```

In Python 2:

```
class A(object):
    def __init__(self):
        super(A, self).__init__()    # Must specify class and self
```

Python 3's zero-argument `super()` uses the enclosing scope to automatically determine the class and instance — no need to repeat yourself.

Slide 18: Dynamic `super()` with Arguments

When you need explicit control:

```
class Base:
    def method(self):
        print("Base.method")

class Mixin:
    def method(self):
        print("Mixin.method")
```

```

class Child(Mixin, Base):
    def method(self):
        super().method()           # Follows MRO

# Manually calling a specific parent:
child = Child()
super(Child, child).method()       # Child's parent in MRO
super(Mixin, child).method()       # Mixin's parent in MRO => Base.method

# This is rarely needed but powerful for debugging.

```

Slide 19: Private Members in Inheritance

Name mangling for pseudo-private attributes:

```

class Parent:
    def __init__(self):
        self.__secret = "hidden"    # Name-mangled to _Parent__secret
        self.public = "visible"

class Child(Parent):
    def reveal(self):
        # print(self.__secret)       # AttributeError!
        print(self._Parent__secret)  # Works - "hidden"
        print(self.public)           # Works - "visible"

c = Child()
c.reveal()

```

- Double underscore prefix triggers **name mangling**
- Prevents accidental override in subclasses
- Not truly private — still accessible via `_ClassName__attr`

Slide 20: `isinstance()` and `issubclass()`

Checking inheritance relationships:

```

class Animal: pass
class Dog(Animal): pass
class Cat(Animal): pass

d = Dog()
print(isinstance(d, Dog))    # True
print(isinstance(d, Animal)) # True (Dog is an Animal)
print(isinstance(d, Cat))    # False

```

```
print(issubclass(Dog, Animal)) # True
print(issubclass(Cat, Animal)) # True
print(issubclass(Dog, Cat))    # False
```

- `isinstance(obj, Class)` — checks if `obj` is an instance of `Class` (or subclass)
- `issubclass(Sub, Parent)` — checks inheritance relationship

Slide 21: Abstract Base Classes (ABC)

Enforcing method implementation in subclasses:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    # Abstract base class
    @abstractmethod
    def area(self):
        # Abstract method – MUST override
        pass

    @abstractmethod
    def perimeter(self):
        # Abstract method – MUST override
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

    def perimeter(self):
        return 2 * 3.14 * self.radius

# s = Shape()      # TypeError: Can't instantiate abstract class
c = Circle(5)      # OK – all abstract methods implemented
```

Slide 22: Mixins — Composable Behavior

Reusing functionality across unrelated classes:

```
class TimestampMixin:
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.created_at = "2024-01-01" # Simplified

    def get_age(self):
        return f"Created on {self.created_at}"
```

```

class LogMixin:
    def log(self, message):
        print(f"[LOG] {self.__class__.__name__}: {message}")

class User(TimestampMixin, LogMixin):
    def __init__(self, name, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.name = name

u = User("Alice")
u.log("User created")           # From LogMixin
print(u.get_age())             # From TimestampMixin

```

Mixins provide **horizontal code reuse** without deep inheritance chains.

Slide 23: Complete Example — Employee Hierarchy

```

class Employee:
    def __init__(self, emp_id, name, salary):
        self.emp_id = emp_id
        self.name = name
        self.salary = salary

    def calculate_pay(self):
        return self.salary

    def __str__(self):
        return f"{self.emp_id}: {self.name}"

class HourlyEmployee(Employee):
    def __init__(self, emp_id, name, hourly_rate, hours_worked):
        super().__init__(emp_id, name, 0)
        self.hourly_rate = hourly_rate
        self.hours_worked = hours_worked

    def calculate_pay(self):           # Override
        return self.hourly_rate * self.hours_worked

class SalariedEmployee(Employee):
    def __init__(self, emp_id, name, annual_salary):
        super().__init__(emp_id, name, annual_salary)

    def calculate_pay(self):           # Override
        return self.salary / 12      # Monthly pay

```

Slide 24: Polymorphism in Action

Same interface, different behavior:

```
employees = [  
    HourlyEmployee("E001", "Alice", 20, 160),  
    SalariedEmployee("E002", "Bob", 60000),  
    HourlyEmployee("E003", "Charlie", 25, 120)  
]  
  
for emp in employees:  
    print(f"{emp.name}: ${emp.calculate_pay():.2f}")  
    # Alice: $3200.00  
    # Bob: $5000.00  
    # Charlie: $3000.00
```

Each subclass provides its own `calculate_pay()` implementation — this is **runtime polymorphism**.

Slide 25: `__init_subclass__` Hook

Reacting when a class is subclassed:

```
class RegistryBase:  
    registry = {}  
  
    def __init_subclass__(cls, **kwargs):  
        super().__init_subclass__(**kwargs)  
        RegistryBase.registry[cls.__name__] = cls  
  
class PluginA(RegistryBase):  
    pass  
  
class PluginB(RegistryBase):  
    pass  
  
print(RegistryBase.registry)  
# {'PluginA': <class '__main__.PluginA'>,  
#   'PluginB': <class '__main__.PluginB'>}
```

- Automatically registers every subclass
 - No decorator or manual registration needed
-

Slide 26: Slots and Inheritance

Memory optimization with `__slots__`:

```
class Base:  
    __slots__ = ['x', 'y']  
  
    def __init__(self, x, y):
```

```

        self.x = x
        self.y = y

class Derived(Base):
    __slots__ = ['z']          # Must add own slots

    def __init__(self, x, y, z):
        super().__init__(x, y)
        self.z = z

d = Derived(1, 2, 3)
print(d.x, d.y, d.z)  # 1 2 3
# d.extra = 5         # AttributeError: no __dict__

```

- `__slots__` prevents `__dict__` creation (saves memory)
- Subclasses must declare their own `__slots__`
- Cannot add new attributes dynamically

Slide 27: Inheritance and `@property`

Overriding properties:

```

class Rectangle:
    def __init__(self, width, height):
        self._width = width
        self._height = height

    @property
    def area(self):
        return self._width * self._height

class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)

    @property
    def area(self):          # Override property
        return self._width ** 2

s = Square(5)
print(s.area)              # 25

```

Properties can be overridden just like regular methods.

Slide 28: Common Pitfall — Forgetting `super()`

Missing parent constructor call:

```
class Parent:
    def __init__(self):
        self.data = "Important data"

class Child(Parent):
    def __init__(self, extra):
        # Forgot: super().__init__()
        self.extra = extra

c = Child("test")
print(c.extra)      # "test"
# print(c.data)     # AttributeError! 'Child' has no attribute 'data'
```

- Parent's `__init__` never ran
- Instance variables from the parent don't exist
- Always call `super().__init__()` unless you intentionally want to skip it

Slide 29: Common Pitfall — Method Name Collisions

Accidental override:

```
class Database:
    def save(self):
        print("Saving to database")

class Logger:
    def save(self):
        print("Saving log")

class User(Database, Logger): # Order matters!
    pass

u = User()
u.save() # "Saving to database" — Database's save() wins
```

- Method with same name in multiple parents = MRO decides which runs
- Change parent order to change behavior: `class User(Logger, Database):`
- Better: use different method names or mixins with care

Slide 30: Composition vs Inheritance

When to use which:

```
# Inheritance: "is-a"
class Car(Vehicle): # Car IS-A Vehicle
```

```

    pass

# Composition: "has-a"
class Car:
    def __init__(self):
        self.engine = Engine()    # Car HAS-A Engine
        self.wheels = [Wheel() for _ in range(4)]

class Engine:
    def start(self):
        print("Engine started")

class Wheel:
    def rotate(self):
        print("Wheel rotating")

```

Inheritance	Composition
"is-a" relationship	"has-a" relationship
Tight coupling	Loose coupling
Code reuse via extension	Code reuse via delegation
Better for hierarchies	More flexible

Note: Modern Python uses `@dataclass` to eliminate manual `__init__` boilerplate in composition models — fields are declared once, and `__init__`, `__repr__`, `__eq__` are auto-generated.

Slide 31: Best Practices

Do's:

- ✓ Favor composition over inheritance when possible
- ✓ Use `super().__init__()` consistently
- ✓ Keep inheritance hierarchies shallow (≤ 3 levels)
- ✓ Use ABCs for defining interfaces
- ✓ Use Mixins for cross-cutting concerns
- ✓ Call `super().__init__()` in every `__init__`

Don'ts:

- ✗ Deep inheritance chains (more than 3 levels)
- ✗ Diamond problems without understanding MRO
- ✗ Forgetting to call `super().__init__()`
- ✗ Using inheritance just to reuse one method
- ✗ Overriding private mangled attributes

Slide 32: Summary

Inheritance:

- Child class acquires parent's attributes and methods
- Supports single, multiple, and multilevel inheritance
- `class Child(Parent):` syntax

Overriding:

- Redefine parent methods in child
- Call parent version with `super()`
- Enables runtime polymorphism

Constructor Call Order:

- `Child.__init__` → calls `super()` → `Parent.__init__` → chain continues
- In multiple inheritance, follows MRO (C3 linearization)
- MRO: child first, then parents in declaration order, `object` last

Key Takeaway: Inheritance is a powerful tool for code reuse, but should be used thoughtfully. Understand MRO, always call `super()`, and prefer composition when the relationship is "has-a" rather than "is-a."

Slide 33: Resources & Further Reading

Official Documentation:

- Python Inheritance: docs.python.org/3/tutorial/classes.html#inheritance
- MRO: docs.python.org/3/howto/mro.html
- ABC Module: docs.python.org/3/library/abc.html
- `super()`: docs.python.org/3/library/functions.html#super

Topics for Further Study:

- Metaclasses and custom MRO
 - Cooperative multiple inheritance patterns
 - `__slots__` in depth
 - Abstract Base Classes vs Protocols (PEP 544)
 - Dependency injection vs inheritance
-

Thank You!

Questions?